Gradient descent and the Gauss–Newton method

Computational Physics II · Unit 3: Maxima and minima of functions

Sebastián

October 2026

Physics Program, Universidad de Pamplona

Outline

The problem

Gradient descent

Newton's method

Two variables

The Gauss–Newton method

Example: a cooling cup of coffee

Wrapping up

The problem

What does it mean to minimize?

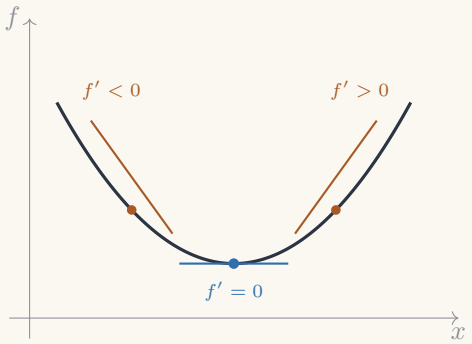
Minimizing means finding the value of x where $f(x)$ is as small as possible.

In physics this comes up all the time:

- **Equilibrium.** A marble in a bowl settles where the potential energy is lowest.
- **Data fitting.** The curve that best describes a set of measurements is the one with the smallest error. This is the problem Gauss–Newton was designed for.
- **Neural networks.** Training one means minimizing a function of a huge number of variables, and it is done with variants of gradient descent.

Both methods we will see today work the same way at heart: start at some point and keep stepping downhill until you cannot go any lower.

Why not just differentiate and set it to zero?



At the bottom of the valley the tangent is horizontal: $f'(x^*) = 0$.

The trouble is that we can rarely solve for x:

- Some equations have no closed-form solution, such as
$$2x - e^{-x} = 0.$$
- Sometimes f comes from a simulation and there is no formula.
- With many variables it is not practical.

So we do what we did with Newton's method in Unit 2: start from an approximation and keep improving it.

Gradient descent

The idea

Picture someone walking down a mountain in thick fog. They cannot see the valley; they can only feel which way the ground slopes, so they take a step downhill and repeat.

In one variable this reads:

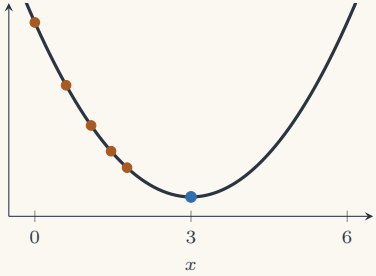
$$x_{k+1} = x_k - \underbrace{\gamma}_{\text{step size}} \underbrace{f'(x_k)}_{\text{slope}}$$

- The minus sign makes us always go downhill: if $f'(x_k) > 0$ we move left, and if $f'(x_k) < 0$, right.
- We choose $\gamma > 0$ ourselves. In machine learning it is called the *learning rate*.
- Where the slope is large, the step is large. Near the minimum $f'(x_k) \approx 0$, so the steps shrink on their own.

A worked example

Take $f(x) = (x-3)^2 + 1$, with $f'(x) = 2(x-3)$ and minimum at $x^* = 3$. We start at $x_0 = 0$ with $\gamma = 0.1$.

k	x_k	$f'(x_k)$	$-\gamma f'(x_k)$	$\lvert x_k - 3 \rvert$
0	0	-6	$+0.6$	3
1	0.6	-4.8	$+0.48$	2.4
2	1.08	-3.84	$+0.384$	1.92
3	1.464	-3.072	$+0.307$	1.536
4	1.771	$\cdots$	$\cdots$	1.229



At each step the distance to the minimum is multiplied by 0.8. It gets there, but slowly: after 20 steps we are still 0.035 away.

The step size

Subtracting 3 from both sides of the rule:

$$x_{k+1} - 3 \;=\; (x_k - 3) - \gamma \cdot 2(x_k - 3) \;=\; (1 - 2\gamma)\,(x_k - 3).$$

γ	$1 - 2\gamma$	What happens
0.1	0.8	goes down slowly
0.5	0	gets there in one step
0.8	-0.6	zigzags, but closes in
1	-1	oscillates between 0 and 6
1.1	-1.2	moves further away each time

It converges if $|1 - 2\gamma| < 1$, that is, if $0 < \gamma < 1$. The 2 is the second derivative, so in general:

$$0 \;<\; \gamma \;<\; \frac{2}{f''(x^*)}$$

In a very narrow valley (large curvature) we need short steps.

The weak spot of the method: we have to choose γ, and the best value depends on the curvature, which the method does not use.

Newton's method

Using the curvature

If we know f'', we can compute the step instead of guessing it. We replace f by its Taylor parabola and jump to its vertex:

$$f(x_k + h) \approx f(x_k) + f'(x_k)\,h + \tfrac{1}{2} f''(x_k)\,h^2 \quad \Longrightarrow \quad f'(x_k) + f''(x_k)\,h = 0.$$

$$x_{k+1} \;=\; x_k - \frac{f'(x_k)}{f''(x_k)}$$

- It is the Newton–Raphson method from Unit 2 applied to $g(x) = f'(x)$, because we are looking for where $f'(x) = 0$.
- If f is a parabola, it gets there in one step. That is why $\gamma = 0.5 = 1/f''$ was the perfect step earlier.

About the name

Newman calls this formula the *Gauss–Newton method*. In most optimization texts that name is reserved for least squares (Section 5).

Newton versus gradient descent

Let us try a function that is not a parabola: $f(x) = \dfrac{x^4}{4} - x$, with minimum at $x^* = 1$. Both start at $x_0 = 2$.

k	Newton	Gradient ($\gamma = 0.2$)
0	2	2
1	1.4167	0.6
2	1.1105	0.757
3	1.0106	0.870
4	1.0001	0.938
5	1.0000	0.973

- With Newton the errors are $1,\ 0.42,\ 0.11,\ 0.011,\ 0.0001$: each one is roughly the square of the previous one (**quadratic** convergence).
- With gradient descent the error is multiplied by a fixed factor, about 0.4 (**linear** convergence). The first step also overshoots the minimum.
- The price: Newton needs f'', and far from the minimum it can head to a maximum or diverge.

Gradient descent is Newton's method with $1/f''(x_k)$ replaced by a constant γ.

Two variables

The gradient

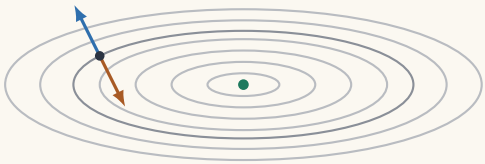
Take $f(x, y) = x^2 + 10y^2$, a long, narrow valley. Differentiating with the other variable held fixed and putting both derivatives in a vector:

$$\nabla f = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right) = (2x,\ 20y).$$

For a small step $\Delta \mathbf{r}$:

$$\Delta f \approx \nabla f \cdot \Delta \mathbf{r} = |\nabla f|\,|\Delta \mathbf{r}| \cos\theta.$$

- $\theta = 0$: we climb as fast as possible.
- $\theta = 180°$: we descend as fast as possible.
- $\theta = 90°$: no change (level curve).

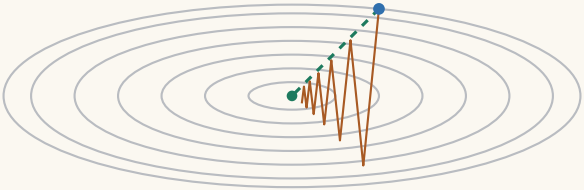


∇f: steepest ascent $\quad -\nabla f$: steepest descent.

The gradient is perpendicular to the level curves.

$$\mathbf{r}_{k+1} = \mathbf{r}_k - \gamma \nabla f(\mathbf{r}_k)$$

The narrow valley problem



Gradient descent with $\gamma = 0.09$ Newton: a single step Start at $(1, 1)$

Each direction has its own curvature: 2 in x and 20 in y. With a single γ,

$$x_{k+1} = (1 - 2\gamma)\, x_k, \qquad y_{k+1} = (1 - 20\gamma)\, y_k.$$

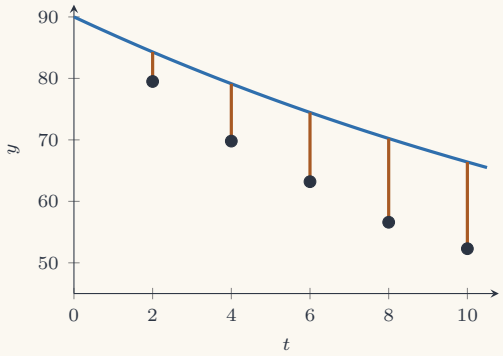
To keep y from diverging we need $\gamma < 2/20 = 0.1$, and then x barely moves.

With $\gamma = 0.09$: the factor is 0.82 in x and -0.8 in y, so y flips sign at every step and the path bounces between the walls.

The ratio of the largest to the smallest curvature is the **condition number** (here, 10). The larger it is, the slower gradient descent becomes.

The Gauss–Newton method

Fitting a model to data



Dots: measurements. Curve: model with a guessed parameter.

Segments: residuals.

We have data (t_i, y_i) and a model $\phi(t; \beta)$ with an unknown parameter β.

Residual (how far off the model is):

$$r_i(\beta) = \phi(t_i; \beta) - y_i$$

Sum of squares:

$$S(\beta) = \sum_{i=1}^{m} r_i(\beta)^2$$

We square the residuals so that errors of opposite sign do not cancel out.

Fitting = finding the β that minimizes S.

The idea behind Gauss–Newton

1. Sensitivity. How much the model changes at each data point if we move β:

$$J_i = \frac{\partial \phi(t_i;\beta)}{\partial \beta}.$$

2. Linearize. Near the current value we treat the model as a straight line:

$$r_i(\beta + \Delta) \approx r_i + J_i\,\Delta.$$

3. This leaves a parabola in Δ:

$$S(\Delta) \approx \sum_i \left(r_i + J_i\,\Delta\right)^2.$$

4. Its minimum. Differentiate and set to zero:

$$\frac{dS}{d\Delta} = 2\sum_i \left(r_i + J_i\,\Delta\right) J_i = 0$$

$$\sum_i J_i r_i + \Delta \sum_i J_i^2 = 0$$

$$\Delta = -\frac{\sum_i J_i\, r_i}{\sum_i J_i^2}$$

Then $\beta \leftarrow \beta + \Delta$, and we repeat because the model is not really a straight line.

Why is it called Gauss–Newton?

If we apply Newton to $S(\beta)$, the step would be $\Delta = -S'/S''$, with

$$S' = 2\sum_i J_i\, r_i,$$

$$S'' = 2\sum_i J_i^2 \;+\; \underbrace{2\sum_i r_i\, \frac{\partial^2 \phi_i}{\partial \beta^2}}_{\text{neglected}}.$$

Dropping the last term:

$$\Delta \approx -\frac{2\sum_i J_i r_i}{2\sum_i J_i^2} = -\frac{\sum_i J_i r_i}{\sum_i J_i^2},$$

which is the same formula.

Gauss–Newton is Newton's method with the second derivative approximated using first derivatives only.

Why can we neglect it?

It is multiplied by the residuals, which are small when the model describes the data well.

When does it fail?

When the starting value is far off, when the model is poor, or when two parameters do almost the same thing.

More than one parameter

If the model has n parameters $\boldsymbol{\beta} = (\beta_1, \ldots, \beta_n)$, the sensitivity becomes an $m \times n$ matrix (one row per data point, one column per parameter):

$$J_{ij} = \frac{\partial \phi(t_i; \boldsymbol{\beta})}{\partial \beta_j}.$$

The same reasoning leads to the *normal equations*:

$$\left(J^\mathsf{T} J\right) \boldsymbol{\Delta} \;=\; -\, J^\mathsf{T} \mathbf{r}$$

- With a single parameter, $J^\mathsf{T} J = \sum_i J_i^2$ and $J^\mathsf{T} \mathbf{r} = \sum_i J_i r_i$: the previous formula.
- Each step requires solving an $n \times n$ linear system, using what we saw in Unit 1 (Gaussian elimination, LU).

Example: a cooling cup of coffee

Setting up the problem

We pour a coffee at $90°$C in a room at $30°$C and measure its temperature every two minutes. By Newton's law of cooling,

$$T(t) = T_a + (T_0 - T_a)\, e^{-kt}$$

with $T_a = 30°$C and $T_0 = 90°$C known. The only unknown is k (in min^{-1}), which tells us how fast it loses heat.

The sensitivity with respect to k is

$$J_i = \frac{\partial T}{\partial k}(t_i) = -\,t_i\,(T_0 - T_a)\, e^{-kt_i}.$$

We want the k that minimizes $S(k) = \sum_i \big(T(t_i) - T_i\big)^2$.

t (min)	T_i (°C)
2	79.5
4	69.8
6	63.2
8	56.6
10	52.3

Simulated data with a little noise, made for this talk.

First iteration by hand

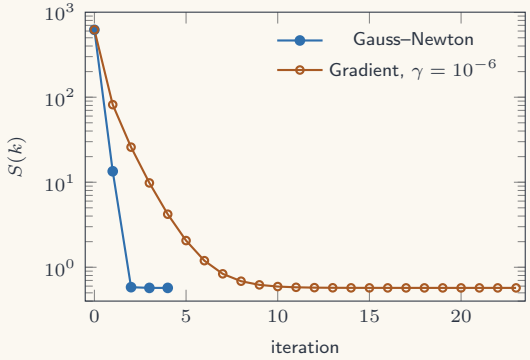
We start from $k_0 = 0.05$ min^{-1}.

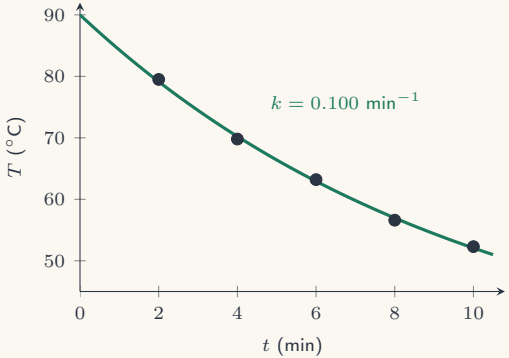
t_i	T_i	$T(t_i)$	r_i	J_i	$J_i\, r_i$	J_i^2
2	79.5	84.29	4.79	-108.6	-520.1	11 790
4	69.8	79.12	9.32	-196.5	$-1\,832.1$	38 610
6	63.2	74.45	11.25	-266.7	$-3\,000.1$	71 126
8	56.6	70.22	13.62	-321.8	$-4\,382.0$	103 525
10	52.3	66.39	14.09	-363.9	$-5\,128.3$	132 437
				$\sum$	$-14\,862.6$	357 488

$$\Delta = -\frac{-14\,862.6}{357\,488} = 0.0416 \qquad \Longrightarrow \qquad k_1 = 0.05 + 0.0416 = 0.0916 \text{ min}^{-1}$$

All the residuals are positive: the model was cooling too slowly, so k had to go up. That is why Δ comes out positive.

Results





Both reach $k \approx 0.100$ min^{-1}: Gauss–Newton in 4 steps and gradient descent in 23. The sum of squares drops from 620 to 0.57; it does not reach zero because the data are noisy.

How sensitive is gradient descent to γ?

γ	Result
1×10^{-6}	23 steps
3×10^{-6}	6 steps
5×10^{-6}	30 steps, oscillating
7×10^{-6}	does not converge

This matches what we saw in one variable, using $S'' \approx 2\sum_i J_i^2 \approx 3.4 \times 10^5$:

$$\gamma_{\text{ideal}} \approx \frac{1}{S''} \approx 2.9 \times 10^{-6},$$

$$\gamma_{\text{max}} \approx \frac{2}{S''} \approx 5.9 \times 10^{-6}.$$

Physical interpretation

With $k \approx 0.100$ min^{-1}, the difference between the coffee and the room halves every

$$t_{1/2} = \frac{\ln 2}{k} \approx 6.9 \text{ min.}$$

The final residuals are about $\pm 0.4^\circ$C, on the order of the measurement noise.

Gauss–Newton does not need us to choose anything: it computes its own step.

Python code (1/2): data and model

```
import numpy as np

# Simulated data (coffee in a room at 30 C)
t = np.array([2, 4, 6, 8, 10.0])              # min
T = np.array([79.5, 69.8, 63.2, 56.6, 52.3])  # C
T_amb, T0 = 30.0, 90.0                        # C

def model(k):  # Newton's law of cooling
    return T_amb + (T0 - T_amb)*np.exp(-k*t)

def residuals_and_J(k):
    r = model(k) - T                    # residuals
    J = -t*(T0 - T_amb)*np.exp(-k*t)    # dT/dk
    return r, J
```

- `np.exp(-k*t)` evaluates all five times at once, with no loop.
- `r` holds the residuals r_i.
- `J` is the sensitivity $J_i = \partial T/\partial k$, worked out by hand.
- Both methods use these two functions.

Python code (2/2): the two methods

```python
# Gradient descent
k, gamma = 0.05, 1e-6
for i in range(100):
    r, J = residuals_and_J(k)
    step = -gamma * 2*np.sum(J*r)       # -gamma*dS/dk
    k += step
    if abs(step) < 1e-6: break
print("Gradient:     k =", round(k, 4), "in", i+1, "steps")

# Gauss-Newton
k = 0.05
for i in range(100):
    r, J = residuals_and_J(k)
    step = -np.sum(J*r) / np.sum(J**2)  # Gauss-Newton
    k += step
    if abs(step) < 1e-6: break
print("Gauss-Newton: k =", round(k, 4), "in", i+1, "steps")
```

Output:

```
Gradient:     k = 0.1 in 23 steps
Gauss-Newton: k = 0.1 in 4 steps
```

- The two loops are identical except for the `step` line.
- They stop when the step is smaller than 10^{-6}.

Wrapping up

Levenberg–Marquardt: a blend of the two

Levenberg (1944) and Marquardt (1963) proposed adding a number $\mu \geq 0$ to the denominator:

$$\Delta = -\frac{\sum_i J_i\, r_i}{\sum_i J_i^2 + \mu}$$

- If $\mu = 0$, it is Gauss–Newton.
- If μ is very large, $\Delta \approx -\dfrac{1}{\mu}\sum_i J_i r_i = -\dfrac{1}{2\mu}\, S'(\beta)$: a short gradient descent step with $\gamma = 1/(2\mu)$.
- If a step improves the fit, μ goes down; otherwise the step is rejected and μ goes up. Far from the solution it acts like gradient descent; close to it, like Gauss–Newton.

It is what SciPy's `curve_fit` uses by default:

```
from scipy.optimize import curve_fit
f = lambda t, k: 30 + 60*np.exp(-k*t)
k_opt, cov = curve_fit(f, t, T, p0=[0.05])   # k_opt = 0.1000 +/- 0.0009
```

Comparison

	Gradient descent	Newton	Gauss–Newton
Problem	minimize any f	minimize any f	minimize a sum of squares
Rule	$x - \gamma f'(x)$	$x - f'(x)/f''(x)$	$\beta - \sum J_i r_i / \sum J_i^2$
Needs	f' and a choice of γ	f' and f''	first derivatives of the model
Convergence	linear	quadratic (near the minimum)	nearly quadratic if the fit is good
Weakness	choosing γ; zigzag in narrow valleys	expensive; unstable far from the minimum	unstable far away; fails with poor models

A bit of history

- **1805.** Legendre publishes the method of least squares and gives it its name.
- **1809.** Gauss publishes it with a probabilistic justification (normally distributed errors) and claims he had been using it since 1795. Legendre disputed the priority.
- **1847.** Cauchy proposes the gradient method, also for orbit calculations.
- **1944 and 1963.** Levenberg and Marquardt propose the blend that bears their names.

Conclusions

- Both methods move toward the minimum step by step. The difference is how much information they use to decide each step.
- Gradient descent only needs the first derivative. It is simple, but slow in narrow valleys, and its behavior depends heavily on γ.
- Gauss–Newton takes advantage of the sum-of-squares structure to approximate the curvature with first derivatives. That is why it converges so fast when the model describes the data well.
- In practice, data fitting uses Levenberg–Marquardt, which combines both.

References

- M. Newman, *Computational Physics*, CreateSpace, 2013. Section 6.4.
- S. C. Chapra and R. P. Canale, *Numerical Methods for Engineers*, McGraw-Hill.
- J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed., Springer, 2006. Chapters 3 and 10.
- W. H. Press *et al.*, *Numerical Recipes*, 3rd ed., Cambridge University Press, 2007. Section 15.5.
- S. M. Stigler, *The History of Statistics*, Harvard University Press, 1986.
- K. Levenberg, "A method for the solution of certain non-linear problems in least squares", *Quarterly of Applied Mathematics* 2 (1944) 164–168.
- D. W. Marquardt, "An algorithm for least-squares estimation of nonlinear parameters", *Journal of the SIAM* 11 (1963) 431–441.

Questions?